```sql
-- Query 1
SELECT
    ListenerID,
    plan_type,
    monthly_fee,
    start_date,

    start_date - LAG(start_date) OVER (
        ORDER BY start_date
    ) AS days_since_previous,

    monthly_fee - LAG(monthly_fee) OVER (
        ORDER BY start_date
    ) AS fee_change,

    LEAD(plan_type) OVER (
        ORDER BY start_date
    ) AS next_plan_type

FROM Subscription
ORDER BY start_date;

-- Query 2
SELECT
    genre_name,
    title,
    duration,

    NTILE(4) OVER (
        ORDER BY duration DESC, TrackID
    ) AS length_quartile,

    FIRST_VALUE(duration) OVER (
        PARTITION BY genre_name
        ORDER BY duration DESC, TrackID
    ) - duration AS minutes_shorter_than_longest

FROM Track
ORDER BY
    genre_name,
    duration DESC,
    title;

-- Query 3
```

```
WITH ranked_plays AS (
    SELECT
        l.ListenerID,
        l.name,
        t.title,
        p.rating,

        ROW_NUMBER() OVER (
            PARTITION BY l.ListenerID
            ORDER BY
                p.rating::INTEGER DESC,
                t.title
        ) AS rn

    FROM Listener l
    JOIN Plays p
        ON l.ListenerID = p.ListenerID
    JOIN Track t
        ON p.Track_id = t.TrackID
)

SELECT
    ListenerID,
    name,
    title,
    rating
FROM ranked_plays
WHERE rn = 1
ORDER BY ListenerID;

-- Query 4
-- create function
CREATE OR REPLACE FUNCTION listener_tier(l_id INTEGER)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
DECLARE
    play_count INTEGER;
    has_subscription BOOLEAN;
BEGIN

    SELECT COUNT(*)
    INTO play_count
    FROM Plays
```

```
    WHERE ListenerID = l_id;

    SELECT EXISTS (
        SELECT 1
        FROM Subscription
        WHERE ListenerID = l_id
    )
    INTO has_subscription;

    IF play_count = 0 THEN
        RETURN 'INACTIVE';

    ELSIF play_count >= 2 AND has_subscription THEN
        RETURN 'LOYAL';

    ELSIF play_count >= 2 THEN
        RETURN 'ACTIVE';

    ELSE
        RETURN 'CASUAL';
    END IF;

END;
$$;

-- every listener and tier
SELECT
    ListenerID,
    name,
    listener_tier(ListenerID) AS tier
FROM Listener
ORDER BY ListenerID;

-- count listeners per tier
SELECT
    listener_tier(ListenerID) AS tier,
    COUNT(*) AS listener_count
FROM Listener
GROUP BY listener_tier(ListenerID)
ORDER BY
    listener_count DESC,
    tier;

-- Query 5
```

```
-- create function
CREATE OR REPLACE FUNCTION genre_summary(g VARCHAR(8))
RETURNS TABLE (
   n_tracks BIGINT,
   avg_duration NUMERIC(10,2),
   total_plays BIGINT,
   top_track VARCHAR(20)
)
LANGUAGE plpgsql
AS $$
BEGIN

   RETURN QUERY
   SELECT
      COUNT(DISTINCT t.TrackID),
      ROUND(AVG(t.duration), 2),
      COUNT(p.ListenerID),
      (
         SELECT t2.title
         FROM Track t2
         LEFT JOIN Plays p2
            ON t2.TrackID = p2.Track_id
         WHERE t2.genre_name = g
         GROUP BY t2.TrackID, t2.title
         ORDER BY COUNT(p2.ListenerID) DESC, t2.title
         LIMIT 1
      )
   FROM Track t
   LEFT JOIN Plays p
      ON t.TrackID = p.Track_id
   WHERE t.genre_name = g;

END;
$$;

-- call it for 'POP'
SELECT *
FROM genre_summary('POP');

-- every genre
SELECT
   g.genre_name,
   gs.n_tracks,
   gs.avg_duration,
```

```
    gs.total_plays,
    gs.top_track
FROM Genre g
CROSS JOIN LATERAL genre_summary(g.genre_name) gs
ORDER BY
    gs.total_plays DESC,
    g.genre_name;

-- Query 6
-- create audit table
CREATE TABLE FeeAudit(
    audit_id SERIAL PRIMARY KEY,
    ListenerID INTEGER,
    old_plan VARCHAR(8),
    new_plan VARCHAR(8),
    old_fee NUMERIC(6,2),
    new_fee NUMERIC(6,2),
    changed_on DATE DEFAULT CURRENT_DATE
);

-- create trigger function
CREATE OR REPLACE FUNCTION audit_fee_change()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN

    IF OLD.monthly_fee <> NEW.monthly_fee THEN

        INSERT INTO FeeAudit(
            ListenerID,
            old_plan,
            new_plan,
            old_fee,
            new_fee
        )
        VALUES (
            OLD.ListenerID,
            OLD.plan_type,
            NEW.plan_type,
            OLD.monthly_fee,
            NEW.monthly_fee
        );
```

```
    END IF;

    RETURN NEW;
END;
$$;

-- after update trigger
CREATE TRIGGER trg_fee_audit
AFTER UPDATE ON Subscription
FOR EACH ROW
EXECUTE FUNCTION audit_fee_change();

-- requests
UPDATE Subscription
SET
    plan_type = 'GOLD',
    monthly_fee = 149.00
WHERE ListenerID = 6;

UPDATE Subscription
SET
    plan_type = 'GOLD',
    monthly_fee = 149.00
WHERE ListenerID = 13;

UPDATE Subscription
SET start_date = '2023-03-20'
WHERE ListenerID = 1;

-- show fee audit
SELECT
    ListenerID,
    old_plan,
    new_plan,
    old_fee,
    new_fee
FROM FeeAudit;

-- Query 7
-- create trigger function
CREATE OR REPLACE FUNCTION set_subscription_fee()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
```

```
BEGIN

    IF NEW.plan_type = 'SILVER' THEN
        NEW.monthly_fee := 99.00;

    ELSIF NEW.plan_type = 'GOLD' THEN
        NEW.monthly_fee := 149.00;

    ELSIF NEW.plan_type = 'PLATINUM' THEN
        NEW.monthly_fee := 199.00;
    END IF;

    RETURN NEW;
END;
$$;

-- create trgger
CREATE TRIGGER trg_set_subscription_fee
BEFORE INSERT ON Subscription
FOR EACH ROW
EXECUTE FUNCTION set_subscription_fee();

-- insert requested rows
INSERT INTO Subscription
VALUES (3, 'PLATINUM', 500.00, '2025-02-01');

INSERT INTO Subscription
VALUES (11, 'SILVER', 0.00, '2025-02-10');

-- show both
SELECT
    ListenerID,
    plan_type,
    monthly_fee,
    start_date
FROM Subscription
WHERE ListenerID IN (3, 11)
ORDER BY ListenerID;

-- Query 8
-- explain analyze
EXPLAIN ANALYZE
SELECT *
FROM PlayLog
```

```
WHERE ListenerID = 42;

-- create b-tree index
CREATE INDEX idx_playlog_listener
ON PlayLog(ListenerID);
-- rerun explain analyze

-- number of rows returned
SELECT COUNT(*) AS rows_returned
FROM PlayLog
WHERE ListenerID = 42;

-- Query 9
-- remove prev index
DROP INDEX idx_playlog_listener;
-- create index
CREATE INDEX idx_playlog_lt
ON PlayLog(ListenerID, Track_id);

-- listener 42
EXPLAIN
SELECT *
FROM PlayLog
WHERE ListenerID = 42;

-- listener 42 + track 137
EXPLAIN
SELECT *
FROM PlayLog
WHERE ListenerID = 42
  AND Track_id = 137;

-- track 137
EXPLAIN
SELECT *
FROM PlayLog
WHERE Track_id = 137;

-- Query 10
-- create hash index
CREATE INDEX idx_playlog_date_hash
ON PlayLog
USING HASH (played_on);
```

```
-- exact date
EXPLAIN
SELECT *
FROM PlayLog
WHERE played_on = '2025-03-15';

-- date range
EXPLAIN
SELECT *
FROM PlayLog
WHERE played_on BETWEEN '2025-03-01' AND '2025-03-07';

-- create B-tree index
CREATE INDEX idx_playlog_date_btree
ON PlayLog(played_on);
-- rerun range query

-- show all playlog indexes
SELECT
    i.indexname,
    i.indexdef,
    pg_size_pretty(
        pg_relation_size(i.indexname::regclass)
    ) AS size
FROM pg_indexes i
WHERE i.tablename = 'playlog'
ORDER BY i.indexname;

-- raw size in bytes
SELECT
    i.indexname,
    i.indexdef,
    pg_relation_size(i.indexname::regclass) AS size_bytes
FROM pg_indexes i
WHERE i.tablename = 'playlog'
ORDER BY i.indexname;
```